

Institución: Universidad Politecnica de Yucatan

Materia: Programacion (2° Cuatrimestre)

Profesor: Jorge Pavia

Alumno: Angel Jair Campos Chale | **Grupo:** A

Fecha: 2026-02-08

HW3 - U3 -Error Handling

HW1-U3 to HW2-U3

HW1 - U3 - Spanish Verb Conjugator — your working Python program with # INPUT, # PROCESS, and # OUTPUT comments.

```

# =====
# INPUT
# =====
# Solicitar al usuario un verbo regular en español que termine en -ar, -er o -ir.
verbo = input("Ingrese verbo: ").strip().lower()

# Estructuras de datos requeridas (adaptadas a español latinoamericano: excluyendo
'vosotros')
pronombres = ['yo', 'tu', 'el', 'nosotros', 'ustedes', 'ellos']
terminaciones = {
    'ar': ['o', 'as', 'a', 'amos', 'an', 'an'],
    'er': ['o', 'es', 'e', 'emos', 'en', 'en'],
    'ir': ['o', 'es', 'e', 'imos', 'en', 'en']
}

# =====
# PROCESS
# =====
# Obtener la raíz (el verbo sin las últimas 2 letras) y el sufijo (las últimas 2
letras)
raiz = verbo[:-2]
sufijo = verbo[-2:]

# Buscar la lista de terminaciones correspondiente en el diccionario
lista_terminaciones = terminaciones[sufijo]

# =====
# OUTPUT
# =====
# Iterar a través de los pronombres e imprimir cada conjugación correspondiente usando
concordancia de índices
for i in range(len(pronombres)):

```

```
conjugacion = raiz + lista_terminaciones[i]  
print(f"{pronombres[i]} {conjugacion}")
```

HW2 - U3 - School Management System: tu programa funcional en Python con comentarios de # INPUT, # PROCESS y # OUTPUT.

```

# =====
# INPUT / DATA STRUCTURES
# =====

materias = ('Matemáticas', 'Programación', 'Inglés')

usuarios = {
    'jperez': {'password': '1234', 'rol': 'alumno', 'nombre': 'Juan Pérez'},
    'amartin': {'password': '1234', 'rol': 'alumno', 'nombre': 'Ana Martín'},
    'cgomez': {'password': '1234', 'rol': 'alumno', 'nombre': 'Carlos Gómez'},
    'lgarza': {'password': '1234', 'rol': 'alumno', 'nombre': 'Lucía Garza'},
    'rtorres': {'password': '1234', 'rol': 'alumno', 'nombre': 'Raúl Torres'},
    'mrodriguez': {'password': '1234', 'rol': 'alumno', 'nombre': 'María Rodríguez'},
    'mlopez': {'password': '1234', 'rol': 'maestro', 'nombre': 'María López'},
    'rgarcia': {'password': '1234', 'rol': 'coordinador', 'nombre': 'Rosa García'}
}

calificaciones = {
    'jperez': {'Matemáticas': 8.5, 'Programación': 9.0, 'Inglés': 7.5},
    'amartin': {'Matemáticas': 9.0, 'Programación': 8.0, 'Inglés': 8.5},
    'cgomez': {'Matemáticas': 7.0, 'Programación': 7.5, 'Inglés': 8.0},
    'lgarza': {'Matemáticas': 9.5, 'Programación': 9.0, 'Inglés': 9.5},
    'rtorres': {'Matemáticas': 6.0, 'Programación': 6.5, 'Inglés': 7.0},
    'mrodriguez': {'Matemáticas': 8.0, 'Programación': 8.5, 'Inglés': 9.0}
}

# =====
# PROCESS & LOGIN FLOW
# =====

usuario_actual = None
rol_actual = None
nombre_actual = None

# Validate login with a while loop (unlimited attempts)
while usuario_actual is None:

```

```

username_input = input("Usuario: ").strip()
password_input = input("Contraseña: ").strip()

if username_input in usuarios and usuarios[username_input]['password'] ==
password_input:
    usuario_actual = username_input
    rol_actual = usuarios[username_input]['rol']
    nombre_actual = usuarios[username_input]['nombre']
    print(f"Bienvenido, {nombre_actual} ({rol_actual})")
else:
    print("Credenciales incorrectas. Intente de nuevo.")

# =====
# BRANCHING BY ROLE
# =====
if rol_actual == 'alumno':
    print(f"\nBoleta de {nombre_actual}")

    aprobadas = set()
    pendientes = set()

    for materia in materias:
        cal = calificaciones[usuario_actual].get(materia, 0.0)
        print(f"{materia}: {cal}")
        if cal >= 8.0:
            aprobadas.add(materia)
        else:
            pendientes.add(materia)

    print(f"Materias aprobadas: {aprobadas}")
    print(f"Materias pendientes: {pendientes}")

elif rol_actual == 'maestro':
    print("\n--- Lista de Alumnos ---")
    for uname, info in usuarios.items():

```

```
        if info['rol'] == 'alumno':
            print(f"- Usuario: {uname} | Nombre: {info['nombre']}")

        alumno_elegido = input("\nAlumno (username): ").strip()
        materia_elegida = input("Materia: ").strip()
        nueva_cal = float(input("Nueva calificación: "))

        if alumno_elegido in calificaciones and materia_elegida in materias:
            calificaciones[alumno_elegido][materia_elegida] = nueva_cal
            print("Calificación actualizada.")
        else:
            print("Error: Alumno o materia inválidos.")

    elif rol_actual == 'coordinador':
        print("\n--- REPORTE DE COORDINACIÓN (Read-Only) ---")

        print("\n1. Lista de Profesores:")
        for uname, info in usuarios.items():
            if info['rol'] == 'maestro':
                print(f"- {info['nombre']} ({uname})")

        print("\n2. Lista de Materias:")
        for materia in materias:
            print(f"- {materia}")

        print("\n3. Lista de Alumnos y Calificaciones:")
        for uname, info in usuarios.items():
            if info['rol'] == 'alumno':
                print(f"\nAlumno: {info['nombre']} ({uname})")
                for materia in materias:
                    cal = calificaciones.get(uname, {}).get(materia, "N/A")
                    print(f"    {materia}: {cal}")
```

Yo, **Angel Campos**, declaro que **NO** he utilizado herramientas de Inteligencia Artificial para la elaboración de este trabajo académico. Afirmo que cuento con evidencias físicas y/o digitales que demuestran mi autoría, incluyendo pero no limitándose a: documentos manuscritos, materiales impresos con anotaciones o subrayado, historial de versiones de documentos electrónicos, o commits en repositorios de código.

Reconozco y acepto que el profesor se reserva el derecho de solicitar dichas evidencias en cualquier momento, especialmente cuando existan sospechas o se detecten conductas que atenten contra la integridad académica, tales como plagio o uso no reportado de herramientas de IA.